

TRENTOS Network Subsystem

Content: • Functional Specification ◦ minimal SEOS ◦ full SEOS • Components • Application API ◦ Basic API ▪ Seos_client_socket_create() ▪ Seos_server_socket_create() ▪ Seos_server_socket_close() ▪ Seos_socket_accept() ▪ Seos_socket_close() ▪ Seos_socket_read() ▪ Seos_socket_write() ▪ Seos_socket_get_info() ▪ Seos_socket_cancelOperation() • Network Stack Core API • Network Interface API • Random Brainstorming notes	Sub Pages: • Network Stack Way Forward - Post picoTCP • NwStack CAMkES Architecture using TAP interface • Proposed new Network API functions
---	---

Functional Specification

minimal SEOS

- support IPv4
- support TCP connections to servers identified by IP address (or hostname if PicoTCP supports this)
- number of possible network connections per application is fixed, system configuration ensures all applications are well known
- socket handles
 - identify a connection
 - are bound to an application
 - can't be shared between different applications
 - are unique integers. On a 64-bit system the handles are 64-bit integers
 - the value is not secret, it is monotonically incremented for each new handle, integer roll over is acceptable.
 - usage of invalid socket handles returns an error
- all socket operations are blocking
- there is no support for timeouts for socket operations
- asynchronous cancellation of all socket operations (not just read or write) is not possible.
- SEOS runtime reports termination of applications (gracefully or crash) to network stack and all open sockets are closed (graceful closing of network connections is not required if an application terminated unexpectedly)
- the socket interface is not (but must be ???) thread-aware, result of parallel calls from different threads is undefined

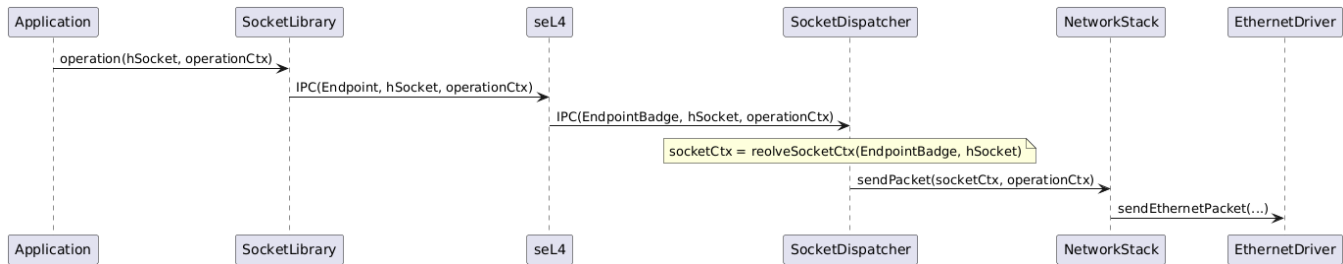
full SEOS

- support TCP and UDP
- provide DNS lookup interface
- support TLS subsystem support being used over the socket interface when proper helpers are implemented
- guarantee number of possible network connections per application. Additional connections are possible if system resources are available
- support for non-blocking sockets
- the socket interface must be thread-safe, calls from different threads are serialized or return an error if this is not possible
- support thread-safe full duplex mode, so reads and writes to a socket are fully independent (within the limits of the underlying protocol)

Components

- socket client library
 - runs in application context and on application stack
 - provides a nice C API
 - hides the IPC mechanism of CAMkES or seL4
- SocketDispatcher
 - optional component, might be part of Network stack or socket client library within the first implementation
 - dispatches socket operations to the actual handler
 - Network Stack
 - TLS Module
- NetworkStack
 - implements TCP/IP
 - implements DNS
- Ethernet Driver

- implements Ethernet and HW access



Application API

- This is the C-Level API that is used by the application. It does not aim for full POSIX compatibility, but a POSIX layer might be implemented on top of this API.
- current development state: https://bitbucket.hensoldt-cyber.systems/projects/SEOS/repos/core-api/browse/seos_nw_api.h?at=SEOS-389
- the API design is agnostic of the actual network subsystem.
 - it can wrap a "common network stack", but may wrap something else also.
 - a socket connection is a generic data channel, the socket handle gives access to it
 - a listener can accept connections from other entities.

Basic API

Seos_client_socket_create()

- function:

```

seos_err_t
Seos_client_socket_create(
    Seos_nw_context ctx,
    seos_nw_client_struct* pClientStruct,
    seos_socket_handle_t* phandle
);
  
```

- Parameters;
 - Seos_nw_context ctx refers to a network subsystem
 - seos_nw_client_struct* pClientStruct defines the type of socket that is to be opened.
 - seos_socket_handle_t* phandle will receive an opaque handle for the connection
- Return codes
 - SEOS_SUCCESS
 - SEOS_ERROR_GENERIC
- Notes
 - open a connection to a server
 - the number of outgoing connections might be limited
 - call blocks until the connection could be established if the connection failed
 - the handle must be closed again
 - terminating the application will close the socket implicitly

Seos_server_socket_create()

- function:

```

seos_err_t
Seos_server_socket_create(
    Seos_nw_context ctx,
    seos_nw_server_struct* pServerStruct,
    seos_nw_server_handle_t* pSrvHandle
);
  
```

- Parameters
 - Seos_nw_context ctx refers to a network interface
 - seos_nw_server_struct* pServerStruct describes the listener
 - seos_nw_server_handle_t* pSrvHandle received an opaque handle to the listener
- Return codes
 - SEOS_SUCCESS

- SEOS_ERROR_GENERIC
- Notes
 - create a listener for incoming connections
 - the number of listeners might be limited
 - the listener handle must be closed via Seos_server_socket_close()
 - terminating the application will close the listener implicitly
 - this call does not block until a connection is established. It returns immediately if the listener could be set up or the setup fails
 - the system could be configured in a way that an application has one or more listeners by default, so Seos_server_socket_create() must not be called but some pre-defined values for pSrvHandle already exist

Seos_server_socket_close()

- function:

```
seos_err_t
Seos_server_socket_create(
    seos_nw_server_handle_t srvHandle
);
```

- Parameters:
 - seos_nw_server_handle_t srvHandle is a handle obtained from Seos_server_socket_create()
- Notes
 - if there are pre-defined srvHandle values from the system, closing them might not be possible. It's up to the implementation to return an error in this case or not

Seos_socket_accept()

- function

```
seos_err_t
Seos_socket_accept(
    seos_nw_server_handle_t srvHandle,
    seos_socket_handle_t *phSocket
);
```

- Parameters:
 - seos_nw_server_handle_t srvHandle is a handle referring to a listener. This is obtained from Seos_server_socket_create() or can be a pre-defined handle provided by the sysdte,
 - seos_socket_handle_t *phSocket receives a handle to socket representing an established connection with a peer
- Return codes
 - SEOS_SUCCESS
 - SEOS_ERROR_GENERIC
- Notes
 - this function is blocking. It may return after a while with an error reporting that there was no connection, so it can be called again.

Seos_socket_close()

- function:

```
seos_err_t
Seos_server_socket_create(
    seos_socket_handle_t handle
);
```

- Parameters:
 - seos_socket_handle_t handle is a socket handle obtained from Seos_server_socket_create() or Seos_socket_accept()
- Return codes
 - SEOS_SUCCESS
 - SEOS_ERROR_GENERIC
- Notes:
 - this function should not block longer than necessary. If closing a connection requires a confirmation from the peer, but this never arrives, there must be rules how long this function will block in the worst case
 - this function should be implemented gracefully to avoid returning errors. If e.g. a socket is already closed, it should not return an error

Seos_socket_read()

- Function

```

seos_err_t
Seos_socket_read(
    seos_socket_handle_t handle,
    void* buf,
    size_t* len
);

```

- Parameters
 - seos_socket_handle_t handle is a handle obtained from various sources
 - Seos_client_socket_create() in case a connection to a peer has been opened
 - Seos_socket_accept() in case a peer has connected
 - it can also be a system defined handle that exists beyond the application's scope
 - void* buf points to a buffer that receives the incoming data.
 - size_t* len contains the size of the buffer on input and the data read into the buffer on return.
- Return codes
 - SEOS_SUCCESS: some data has been read. This code is also returned if the underlying connection has been closed, but there is still unread data that has to be read.
 - SEOS_ERROR_CONNECTION_CLOSED: This error is returned if there are no unread bytes in the internal buffer and the connection has been closed.
 - SEOS_ERROR_GENERIC
- Notes:
 - in general, the call is blocking until up to the requested number of bytes have been read
 - function may return less data than requested in some conditions, e.g.
 - if the connection has been closed before all requested data should be read. The true socket state is then hidden temporarily from the application in this case.
 - simplify the handling of certain connection failure states, where only partial data could still be read
 - connection protocol timeouts
 - handle internal issues, where the call needs to return to unblock the application
 - the API behavior is designed to be convenient for application usage. **ToDo: What is the best approach?**
 - option 1
 - If the function has read at least one byte, it will return SEOS_SUCCESS. Any errors are hidden from the application for now, it may be revealed on the next call.
 - if the function returns an error, then no data has been read and len is set to 0.
 - option 2
 - if an error is returned, the function may also have read some data and len is set to the number of bytes that have been read.
 - pro:
 - requires less API calls to detect errors in application
 - simplify network stack implementation
 - con
 - application error handling gets more complex
 - API behavior for buf/len combinations

buf	len	Behavior
NULL	NULL	probe connection state - SEOS_SUCCESS if connection is open - SEOS_ERROR_CONNECTION_CLOSED if connection is closed - may also return other error codes to indicate status
	0	probe connection state, len returns MIN(len, unread bytes)
	not 0	
not NULL	NULL	SEOS_ERROR_GENERIC
	0	returns immediately with len = 0. Return code can be SEOS_SUCCESS or an error code
	not 0	block to read data, returns - SEOS_SUCCESS and len >= 0 - error code and len = 0

Seos_socket_write()

- Function

```

seos_err_t
Seos_socket_write(
    seos_socket_handle_t handle,
    const void* buf,
    size_t* len
);

```

- Parameters
 - seos_socket_handle_t handle is a handle obtained from various sources
 - Seos_client_socket_create() in case a connection to a peer has been opened
 - Seos_socket_accept() in case a peer has connected
 - it can also be a system defined handle that exists beyond the application's scope
 - const void* buf points to a buffer with data to write
 - size_t* len contains the size of the buffer on input and the data written to the buffer on return.
- Return codes
 - SEOS_SUCCESS: some data was written. This code is also returned if the underlying connection has been closed but some data was written successfully.
 - SEOS_ERROR_CONNECTION_CLOSED: This error is returned if no data was written because the connection is closed
 - SEOS_ERROR_GENERIC
- Notes:
 - blocking call that writes the data
 - in general, there is no guarantee that the data has been received by the peer. This may depend on the underlying protocol.
 - function may return less data than requested in some conditions, e.g. to simplify the handling of certain connection failure states, where some data could still be read. It is send to the application without reporting connection failure. The next read call will return the failure then
 - the API behavior is designed to be convenient for application usage. **ToDo: What is the best approach?**
 - option 1
 - if the function has written at least one byte, it will return SEOS_SUCCESS. Any errors are hidden from the application, but may be returned in the next call.
 - if the function returns an error, then no data has been written and len is set to 0.
 - option 2:
 - if an error is returned, the function may also have written some data and len is set to the number of bytes that have been read.
 - pro:
 - requires less API calls to detect errors in application
 - simplify network stack implementation
 - con
 - application error handling gets more complex
 - API behavior for buf/len combinations

buf	len	Behavior
NULL	NULL	probe connection state - SEOS_SUCCESS if connection is open - SEOS_ERROR_CONNECTION_CLOSED if connection is closed - may also return other error codes to indicate status
	0	SEOS_ERROR_GENERIC
	not 0	
not NULL	NULL	SEOS_ERROR_GENERIC
	0	- block to write data - returns SEOS_SUCCESS and len >= 0 if at least some data could be sent. - may send a datagram of zero length (or without payload) if the underlying protocol supports this - if the connection has been closed then SEOS_ERROR_CONNECTION_CLOSED is returned and no data is sent
	not 0	

Extended API proposal

The extended API collects functions that are currently not implemented, but considered necessary in complex usecases

Seos_socket_get_info()

- obtain information about the state of a socket. e.g.
 - socket type
 - connection peer infos
 - connection state
 - un-read bytes in input buffer
 - un-written bytes in output buffer
 - un-ack'ed bytes

Seos_socket_cancelOperation()

- cancelOperation(socketHandle, flags)
- socketHandle is an optional parameter to limit cancellation to an existing socket.
- flags can be
 - CANCEL_ALL: cancel all operations, socketHandle is ignored
 - CANCEL_LISTEN: cancel listen operations, socketHandle is ignored
- can be used to asynchronously unblock all blocking operations for this socket
- function may not be implemented in the file SEOS version

Network Stack Core API

- This is the internal C-Level API of the network core module, it is called by the seL4/CAMkES specific wrapper layer that receives application API calls.
- The core API is basically the application API, but with an additional parameter that identifies the application that invoked the API
- application triggered functions
 - connect(application, server, port)
 - listen(application, port)
 - close(application, socketHandle)
 - read(application, socketHandle, buffer, len)
 - write(application, socketHandle, buffer, len)
 - cancelOperation(application, socketHandle, flags)
- system triggered functions
 - termination(application)

Network Interface API

- this is the lower level API of the network stack
- currently this API is not specified and uses whatever the network stack implemented defines (e.g. PicoTCP in the current case)
- basically this API defines how to
 - send and receive network packets
 - get link state information

Random Brainstorming notes

- an abstract socket interface could work for every communication channel (TCP, TLS)
- modules implementing a socket interface for I/O, so they can be layered, e.g.
- MQTT uses sockets as lower interface
- TLS connects to socket on lower interface and provides a socket at the upper layer
- IP Stack does lower I/O to driver via Socket interface
- each Channel of the UART bridge is accessed via Socket interface